SOCIAL EAGLE
learn. build. operate

WEEK 2 · DAY 3

Trace a LangChain App with LangSmith

Assignment | learn. build. operate

1. Problem Statement

When a LangChain app gives a bad answer the terminal tells you almost nothing: not the prompt actually sent, not how long it took, not what it cost. **LangSmith** records every run. Build a small **LangChain** app that sends prompts to **OpenAI**, switch tracing on, and watch the runs appear, with no code change.

Tools: Python, LangChain, OpenAI and LangSmith API keys

2. Requirements

- Keep all your code in a single file named `traced_app.py`.
- Use **LangChain** with `ChatOpenAI`, not the OpenAI SDK directly.
- Send at least **3 different prompts** in one run, and print each answer.
- Turn tracing on with **environment variables only**. No tracing code in the file.
- Set a **project name** so your runs land somewhere you named, not in "default".
- Open LangSmith and find your runs.
- For one run, read off the exact prompt sent, the response, the latency and the token counts.

> **Think it through**
>
> Tracing needs no code, which also means it can be silently off. How would you check it is actually on before you go hunting the dashboard for runs that were never sent? And once you can see tokens and latency side by side, which of your three prompts is the expensive one, and is it expensive for a good reason?

3. Steps to complete

These are hints, not the full answer. Fill in the blanks yourself.

Step 1: Set up and get both keys

Sign up at `smith.langchain.com` and create an API key there, alongside your OpenAI one.

```
mkdir traced-app && cd traced-app
python -m venv venv && source venv/bin/activate
pip install langchain langchain-openai

export OPENAI_API_KEY="your-openai-key"
export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="your-langsmith-key"
export LANGSMITH_PROJECT="caie-week2"
```

On Windows PowerShell use `$env:NAME="value"` for each line. The variables are `LANGSMITH_*`, older tutorials show `LANGCHAIN_TRACING_V2`, which is the previous naming.

Step 2: Write the app

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

prompts = ["...", "...", "..."]
for p in prompts:
    print(llm.invoke(p).content)
```

Nothing in this file mentions LangSmith. That is the lesson: the environment variables do all of it.

Step 3: Run it and go looking

```
python traced_app.py
```

Open `smith.langchain.com`, find your project, and open one run. Expand it until you can see the exact text that was sent to the model.

4. Final output

```
Project: caie-week2

run 1   "..."           1.2s    412 tokens
run 2   "..."           0.9s    280 tokens
run 3   "..."           3.4s   1,180 tokens   <- the expensive one
```

5. What to submit

- A screenshot of your LangSmith project with the three runs listed.
- A screenshot of one run opened up, showing the prompt, response, latency and tokens.
- Your `traced_app.py` on **GitHub**, link in **WhatsApp**, video in the **community**.